Lab 3: Component Modelling & Architectural Pattern Selection

NAME : ANIRUDH M

SRN: PES1UG24CS556

CLASS: 5 ‘J’

1. Scenario

Student Project Portfolio & Showcase App

Domain: Campus & Academic Operations
Users: Student Team, Panel Judge

Key Functional Requirements

- Students can create and manage their project showcase profiles.
- Students can upload multimedia project artifacts.
- Students can provide project/demo and GitHub information.
- Panel judges can evaluate submitted projects.
- Users can participate in project voting.
- Voters can be verified to prevent invalid/duplicate voting.
- Project results/leaderboard can be displayed.

Key Non-Functional Requirements

- **Security:** Protect project data and ensure only authorized users perform restricted actions.
- **Performance:** Voting, evaluation and showcase pages should respond quickly.
- **Usability:** Students and judges should be able to use the system easily.
- **Scalability:** The system should handle multiple users accessing projects and voting.

Main Challenges

- **Security:** Prevent unauthorized access and fraudulent/duplicate voting.
- **Performance:** Handle multiple users accessing projects and voting simultaneously.
- **Usability:** Provide a simple interface for students and judges.

- **Scalability:** Support increasing numbers of projects, users and votes.

2. Architectural Style Analysis:

Architecture	How it fits your system	Advantages	Disadvantages
Layered Architecture	Separate Presentation, Business Logic, and Data layers	Clear separation of concerns; easy to understand and maintain	Performance overhead between layers; difficult to scale individual components
Microservices Architecture	Separate showcase, artifact, evaluation, voting and leaderboard functionality into services	Independent scaling; fault isolation; independent deployment	More operational complexity; network communication can introduce latency; data consistency can be challenging
Client-Server Architecture	Student/Judge clients communicate with a centralized server	Centralized control; simpler deployment; easier data consistency	Server can become a bottleneck; single point of failure; limited fault tolerance

Layered Architecture:
The system can be divided into presentation, business logic, and data layers. It provides clear separation of concerns and is easy to understand. However, it may introduce performance overhead and makes it difficult to scale individual components independently.

Microservices Architecture:
The system can divide major functionalities such as project showcase, artifact management, evaluation, voting, and leaderboard into independent services. It supports independent scaling and fault isolation. However, it introduces operational complexity, network latency, and data consistency challenges.

Client-Server Architecture:
Students and judges can interact with a centralized server that manages application data and functionality. It provides centralized control and simpler deployment, but the server may become a scalability bottleneck and a single point of failure.

3. Architecture Selection & Component Identification:

3.1 Select Architecture

Based on your system, I recommend we use:

Microservices Architecture

Put this in your document:

Architecture Selection:

We chose **Microservices Architecture** for the Student Project Portfolio & Showcase App.

Reason 1 — Independent functionality

Your application has several distinct functions:

- Project showcase/profile management
- Multimedia artifact management
- Project evaluation
- Public voting and voter verification
- Leaderboard

These can be separated into independent services. This fits the Lab 3 handout's description of microservices as **small, independently deployable services** with independent scaling and fault isolation.

Reason 2 — Different workloads

Your Lab 1 specifically requires **public voting and leaderboard calculations to update in real time with API response latency under 300 ms**.

Voting/leaderboard functionality may receive much more traffic than project profile management, so separating these functions allows them to be scaled independently.

Security advantage

Microservices allow security controls to be applied separately to sensitive functionality.

For example:

The Voting and Voter Verification Service can enforce authentication, voter verification and duplicate-vote prevention independently from the public showcase functionality.

This directly addresses your Lab 1 requirement for **strict anti-sybil protection**.

Performance benefit

The Voting and Leaderboard services can be independently scaled to handle high numbers of voting requests and maintain fast leaderboard updates without requiring the entire application to scale.

This connects directly to your Lab 1 real-time/under-300-ms requirement.

3.2 Identify the 5 Components

Component 1 — Showcase Profile Service

Handles:

- Creating project profiles
- Abstract/project information
- GitHub repository URL
- Demo/video link
- Publishing project information

Component 2 — Artifact Management Service

Handles:

- Uploading multimedia artifacts
- Managing submitted project files/media
- Retrieving project artifacts

Component 3 — Evaluation Service

Handles:

- Judge access
- Digital rubric
- Project evaluation
- Storing evaluation results

Component 4 — Voting & Voter Verification Service

Handles:

- Voter verification
- Casting votes
- Preventing duplicate/invalid voting
- Recording votes

This combines your Lab 2 stories:

- **SPPSA-7 — Cast Vote**
- **SPPSA-8 — Verify Voter**

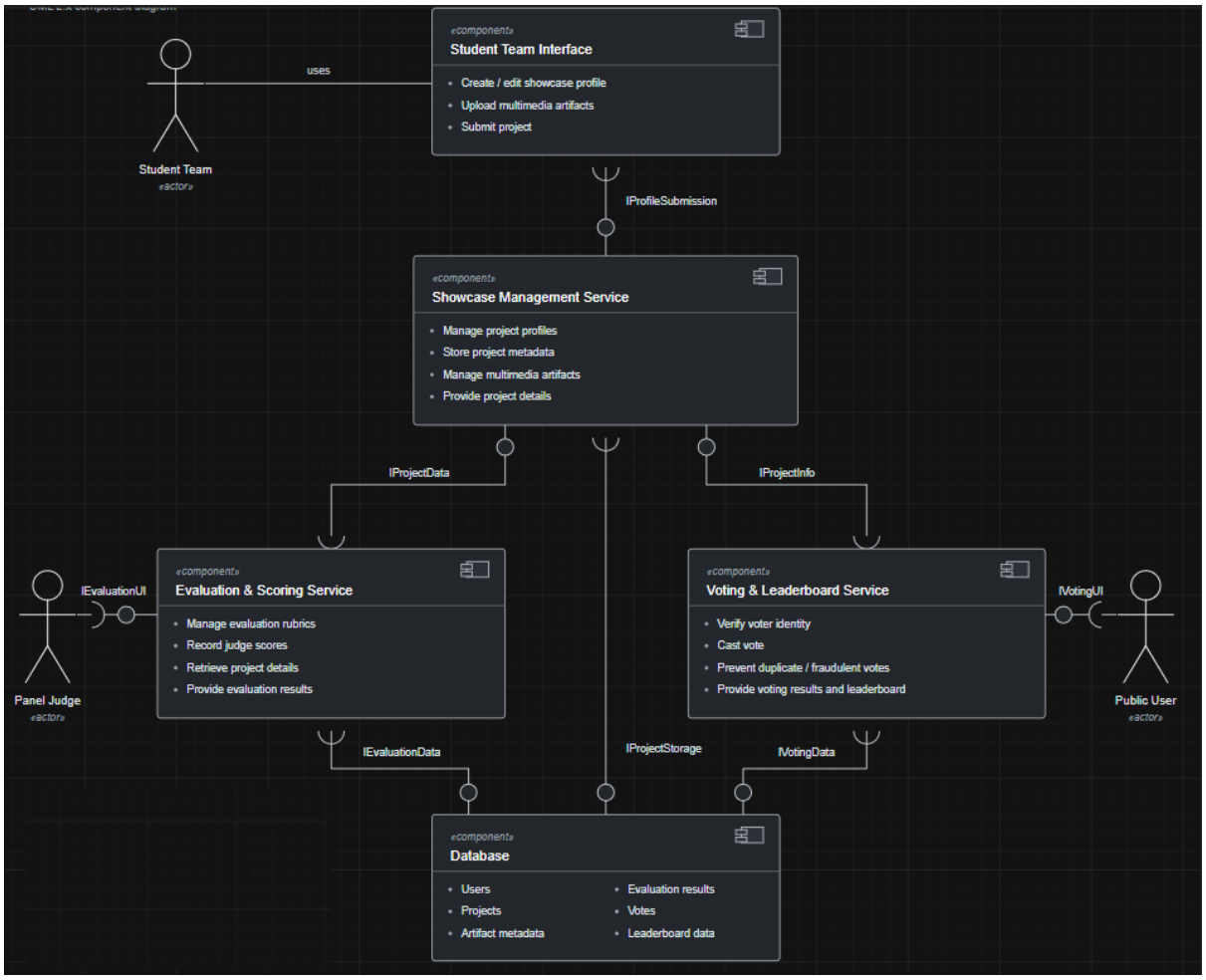
It also directly addresses the anti-sybil requirement in Lab 1.

Component 5 — Leaderboard Service

Handles:

- Calculating project rankings
- Updating voting results
- Providing leaderboard information

4. UML Component Diagram



UML Component Diagram for the Student Project Portfolio & Showcase App

Written Justification

ARCHITECTURAL JUSTIFICATION

Student Project Portfolio & Showcase App

Architecture: Microservices Architecture

1. Architectural Choice

We chose **Microservices Architecture** for the **Student Project Portfolio & Showcase App**.

2. Reason 1 — Independent Functionality

The system contains several distinct functions such as **project showcase/profile management, multimedia artifact management, project evaluation, voting and voter verification, and leaderboard management**. These functionalities can be separated into independent services. This supports independent scaling, fault isolation, and independent deployment, which are characteristics of microservices architecture.

3. Reason 2 — Different Workloads

The application has different workloads. In particular, **public voting and leaderboard calculations** may receive many more requests than project profile management. Lab 1 also specifies that voting and leaderboard calculations should update in real time with an **API response latency under 300 ms**. Separating these functions allows the high-traffic voting functionality to be scaled independently.

4. Security Advantage

Microservices allow security controls to be applied separately to sensitive functionality. For example, the **Voting and Voter Verification Service** can enforce voter verification and duplicate-vote prevention independently from the public showcase functionality. This addresses the Lab 1 requirement for strict **anti-sybil protection** and helps ensure that only authorized actions are performed.

5. Performance Benefit

The **Voting and Leaderboard Service** can be scaled independently when voting traffic increases. This allows the system to handle a large number of voting requests and maintain fast

leaderboard updates without requiring the entire application to be scaled. This supports the Lab 1 requirement for real-time updates and an API response latency below 300 ms.